

AI-Powered Fault Localization Using Vision-Language Models

Describer-Decider Engine — Full Project Report

Table of Contents

1. [Project Overview](#)
 2. [Research Hypothesis and Motivation](#)
 3. [System Architecture](#)
 4. [Data Flow and Pipeline](#)
 5. [Input Data Formats](#)
 6. [Python Modules — Detailed Explanation](#)
 - [6.1 models.py — Data Models and Helper Functions](#)
 - [6.2 agents.py — AI Agents](#)
 - [6.3 graph.py — LangGraph Orchestration](#)
 - [6.4 input_adapter.py — Format Detection and Normalization](#)
 - [6.5 parser/log_parser.py — Unified Log Ingestion](#)
 - [6.6 runner.py — Colab Entry Point](#)
 7. [Agent Prompting Methodology](#)
 - [7.1 Vision Observer Prompt Design](#)
 - [7.2 Logic Decider Prompt Design](#)
 - [7.3 Log Observer Prompt Design](#)
 - [7.4 Root Cause Analysis Agent Prompt Design](#)
 8. [Cascading Failure Analysis](#)
 9. [LLM-Powered Root Cause Chain Analysis](#)
 10. [Suspicious Step Ranking](#)
 11. [Confidence Scoring and Calibration](#)
 12. [Technology Stack and Design Choices](#)
 13. [Evaluation Results](#)
 14. [Known Limitations and Future Work](#)
-

1. Project Overview

This project presents an **AI-powered fault localization system** designed to reduce the manual effort of diagnosing failures in UI-driven applications (web and mobile). The system analyzes sequences of screenshots captured before and after each interaction step, combined with application execution logs, to automatically determine whether each step passed or failed — and when a failure is found, to identify its root cause and causal chain.

The implementation is the **Describer-Decider Engine**, a multi-agent pipeline built on OpenAI's GPT-4o family of models, orchestrated using LangGraph. It includes four specialized AI agents:

- **Vision Observer** — describes what changed visually between PREV and POST screenshots

- **Log Observer** — parses execution logs and extracts structured error evidence
- **Logic Decider** — fuses visual and log evidence to render a pass/fail verdict with confidence
- **Root Cause Analysis Agent** — performs a cross-step retrospective to identify causal chains among all failures

The system has been evaluated on **12 applications / 136 steps** drawn from two independent datasets — 7 proprietary traces and 5 external Group 411 cases — achieving a **combined F1 score of 92%**.

2. Research Hypothesis and Motivation

Core Hypothesis

A Vision-Language Model (VLM) can observe UI state changes between a "before" screenshot (**prev**) and an "after" screenshot (**post**), and a reasoning LLM can determine whether the application behaved correctly given the intended action, producing a structured, machine-readable fault report.

Motivation

Traditional automated UI testing tools produce binary pass/fail outcomes and often stop at the first failure with no further diagnosis. When a multi-step workflow fails, identifying the **root cause** — the first step that went wrong and triggered a cascade — requires manual inspection of logs, screenshots, and test traces.

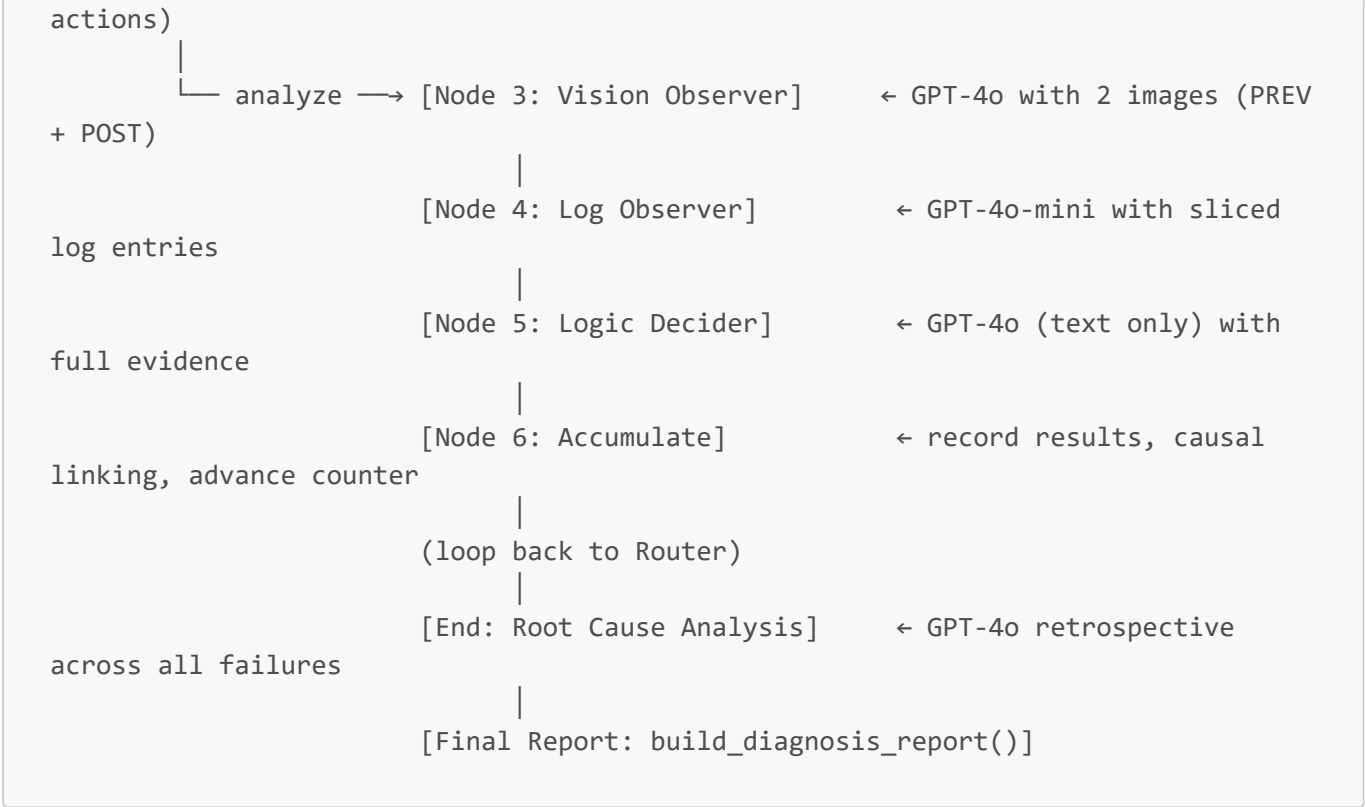
The Describer-Decider system addresses this by:

1. **Separating observation from judgment:** a Vision Observer describes what it sees without making verdicts; a Logic Decider applies structured decision rules to reach a verdict.
2. **Fusing multiple evidence sources:** visual UI changes, application logs, and network events are combined into a unified judgment.
3. **Tracking causal chains:** failures are linked to their upstream causes both through a mechanical scoring algorithm (**find_likely_cause**) and a post-hoc LLM reasoning agent (**run_root_cause_agent**).
4. **Operating without application source code:** the system requires only screenshots, step descriptors, and optionally execution logs.

3. System Architecture

The system uses a **four-agent, six-node pipeline**:





Four AI Agents and Their Roles

Agent	Model	Input	Output
Vision Observer	GPT-4o	PREV + POST screenshots + action context	VisionObservation : element presence, visual changes, screen summary
Log Observer	GPT-4o-mini	Sliced log entries + step context	LogObservation : error flags, network events, anomaly flags
Logic Decider	GPT-4o	VisionObservation + LogObservation + prior failure history	FinalDecision : is_failure, failure_type, confidence, root_cause
Root Cause Analysis	GPT-4o	All step results + all detected errors	RootCauseReport : causal chain links, corrupted state, investigation recommendation

4. Data Flow and Pipeline

Step Lifecycle

Each step in the execution trace goes through the following lifecycle:

1. **Router Node** reads the current step index, extracts its context (action type, target element, instruction), and decides whether to skip (non-VLM step, wait action, missing images) or analyze.
2. **Vision Node** encodes the PREV and POST screenshots as base64 strings and sends them to GPT-4o along with the step context. The model returns a structured description of UI element presence and visual changes between the two screenshots.

3. **Log Node** retrieves log entries relevant to the current step. It checks, in priority order:
- Inline G411 console/network logs embedded in the step dict
 - Pre-assigned log entries from the unified log parser (for `flat_indexed`, `flat_assertions`, or `nested` formats)
 - Dynamically sliced entries from the raw log (keyword scoring + positional fallback)
 - Per-step legacy log files (`step_N_logs.json`)
4. **Decider Node** receives both observations along with the last 3 active failure summaries and applies a priority-ordered rule set to produce a pass/fail verdict with a confidence score.
5. **Accumulate Node** records the step result, applies post-hoc confidence corrections, and — when confidence is high enough — attempts to link the failure to an upstream cause via `find_likely_cause()`.
6. **Root Cause Analysis** runs once after all steps are processed: `run_root_cause_agent()` makes a single GPT-4o call with the complete list of all step results and detected errors, reasoning freely about causal chains including long-gap semantic links that the mechanical scoring missed.
7. **Final Report** is assembled: `build_diagnosis_report()` formats failure chains (mechanical), root cause chain analysis (LLM), suspicious step rankings, and summary statistics.
-

5. Input Data Formats

The system supports two input formats, detected automatically:

v33 Format (Native)

A flat JSON array of step objects, each with:

Field	Description
<code>step_idx</code>	Numeric step identifier; maps to <code>step_N_prev.png</code> / <code>step_N_post.png</code>
<code>action</code>	<code>click</code> , <code>waitUntil</code> , <code>abstractVerification</code> , <code>wait</code>
<code>element</code>	Target UI element string
<code>stepInstruction</code>	Human-readable intent (supports Turkish and English)
<code>requiresVLM</code>	Optional boolean; if <code>false</code> , the step is skipped

For `waitUntil` steps, the real `element` and `stepInstruction` are nested inside a `condition` sub-object. `extract_step_context()` handles this automatically.

G411 Format (External)

Used by Group 411 external test cases:

- Top-level dict with a `stepDetails` list
- Each step carries inline base64 screenshots (`prevScreenshot`, `currScreenshot`)
- May include `consoleLogs` and `networkLogs` as plain text strings

- No explicit `step_idx` field (array index is used)

`input_adapter.py` normalizes G411 input into the canonical v33 format, embedding images and logs as `_inline_*` private fields.

6. Python Modules — Detailed Explanation

6.1 `models.py` — Data Models and Helper Functions

Role: Defines all Pydantic data models used for agent input/output, the LangGraph `GraphState`, and the full suite of helper functions supporting log slicing, context extraction, causal analysis, and report generation.

Pydantic Models

VisionObservation

```
class VisionObservation(BaseModel):
    prev_screen_elements: str # Element-level observations on PREV screen
    post_screen_changes: str # Element-level observations on POST screen
    semantic_screen_summary: str # 1-2 sentence high-level screen state after
    action
```

The structured output of the Vision Observer. Separates observations about the previous state from the post-action state, allowing the Logic Decider to compare them precisely. Defaults to safe "Could not analyze" strings on parse failure.

FinalDecision

```
class FinalDecision(BaseModel):
    expected_value: str # Specific data value expected (for
    abstractVerification)
    observed_value: str # Actual data value seen in POST
    expected_behavior: str # What should have happened
    observed_behavior: str # What actually happened
    is_failure: bool
    failure_type: str # "element_missing" | "action_failed" |
    "content_mismatch"
    # | "system_error" | "none"
    observation: str # One-line verdict summary
    root_cause: str # Direct cause + upstream link if applicable
    confidence: float # 0.0-1.0, self-assessed by Logic Decider
```

The structured verdict. The `expected_behavior` / `observed_behavior` pair provides a behavioral comparison that makes reports human-readable and usable for regression analysis. `confidence` feeds into causal chain gating and report statistics.

ChainLink and **RootCauseReport** (added in the latest version)

```

class ChainLink(BaseModel):
    from_step: int    # Downstream failing step
    caused_by: int    # Upstream causing step

class RootCauseReport(BaseModel):
    chain_links: List[ChainLink]    # All identified causal links
    root_step: int                  # True root of the failure cascade
    chain_summary: str               # 2-3 sentences on what broke and why
    corrupted_state: str             # What app state was corrupted
    downstream_impact: str           # How the corruption propagated
    recommended_investigation: str  # What the developer should check first
    confidence: float                # LLM's certainty about the causal chain

```

Output of the Root Cause Analysis Agent. `chain_links` form a directed acyclic graph that the report renderer traverses using `_resolve_root()` to compute exact hop distances.

Log Models

```

class LogEntry(BaseModel):
    timestamp: str
    level: Literal["info", "warning", "error", "critical"]
    source: Literal["network", "console", "system", "application"]
    message: str

class NetworkEvent(BaseModel):
    method: str; url: str; status_code: int; latency_ms: float; error_message: str

class LogObservation(BaseModel):
    has_logs: bool
    log_summary: str
    relevant_errors: List[LogEntry]    # Up to 5 entries
    network_events: List[NetworkEvent] # Up to 5 entries
    has_network_failure: bool          # Any 4xx/5xx or timeout
    has_system_error: bool              # Any error/critical log entry
    latency_anomaly: bool               # Any request > 5000ms

```

The three boolean flags (`has_network_failure`, `has_system_error`, `latency_anomaly`) allow the Logic Decider to quickly incorporate log evidence without re-parsing the full log text.

GraphState

```

class GraphState(TypedDict):
    # Persistent across all steps
    current_step_index: int
    steps_data: List[dict]
    trace_files_path: str
    final_report: str
    stop_execution: bool

```

```

detected_errors: List[dict]
active_failures: List[str]
active_failure_map: Dict[str, dict] # str(step_idx) → failure dict
step_results: List[dict]
# Transient – overwritten each iteration
_current_ctx: Optional[dict]
_current_vision_obs: Optional[dict]
_current_log_obs: Optional[dict]
_current_decision: Optional[dict]
_route: Optional[str]
_skip_reason: Optional[str]

```

GraphState is the central shared memory of the LangGraph graph. Persistent fields accumulate across all steps; transient fields (prefixed `_`) are overwritten at each iteration by their respective nodes. The **active_failure_map** uses string keys because LangGraph serializes state as JSON, which requires string dict keys.

Key Helper Functions

extract_step_context(raw_item, current_idx)

Normalizes flat and nested step dict formats into a canonical context dict. Handles the **waitUntil** special case where the real **element** and **stepInstruction** are nested inside a **condition** sub-object. Returns **step_num**, **action**, **instruction**, **target**, and **requires_vlm**.

robust_json_extract(text)

Extracts a JSON object from model output that may be wrapped in markdown code blocks or contain Python boolean literals. First tries ````json{...}```` via regex, then falls back to a greedy `{...}` match. Replaces **True/False/None** with JSON-valid equivalents before parsing.

_normalize_keywords(text)

Tokenizes a string for log slicing keyword matching. Strips Turkish diacritical characters (ç→c, ğ→g, ı→i, ö→o, ş→s, ü→u). Preserves decimal/time formats (e.g., **12.15**, **13:35**) by not splitting on dots/colons between digits. Returns a set of lowercase tokens of 3+ characters.

extract_step_logs(ctx, all_logs, step_idx, total_steps)

Two-pass strategy for assigning log entries to a specific step from a unified raw log:

- **Pass 1 (keyword scoring):** Scores each log entry against the step's **target** and **instruction** keywords. Matches in **element_id** get score +2; matches in **message** or **screen** get score +1. **vlm_agent** source entries are excluded to prevent contamination from test-framework assertion records that repeat target text from unrelated steps.
- **Pass 2 (positional window fallback):** When fewer than 3 keyword matches are found, falls back to a proportional positional window based on the step's position in the total step sequence. Window size is **total_log_entries / total_steps** entries.

find_likely_cause(current_idx, current_failure_type, current_ctx, active_failure_map)

Multi-factor causal scoring function. For each prior failure in `active_failure_map`, computes a composite score from three factors:

1. **Element overlap score (0–2.0):** Substring match (current target in prior instruction → 2.0), reverse substring match (prior target in current instruction → 1.5), or keyword intersection (→ 1.0).
2. **Failure type correlation:** A pre-defined `_TYPE_CORRELATION` matrix encodes domain knowledge about which failure types cause others.
3. **Temporal proximity decay:** $1.0 / \max(\text{distance}, 1)$ — failures closer in step index are weighted more strongly.

Final score = $(\text{element_score} + \text{type_score}) * \text{proximity_score}$. Returns the best-scoring step index only if `score > 0.3` AND element overlap > 0. Element overlap is mandatory — type correlation alone is not sufficient.

The `_TYPE_CORRELATION` matrix:

Upstream →	element_missing	content_mismatch	action_failed	system_error
element_missing	1.0	0.6	0.8	0.3
content_mismatch	0.3	1.0	0.5	0.3
action_failed	0.8	0.8	0.9	0.5
system_error	0.5	0.5	0.5	0.7

`build_failure_chains(detected_errors, active_failure_map)`

Groups detected failures into causal chains using the `caused_by_step` field. Builds a parent-of adjacency map, finds chain roots (failures with no upstream cause), and traverses chains breadth-first. Output distinguishes `ISOLATED FAILURE` entries from `FAILURE CHAIN N: (K steps)` blocks with root annotation and downstream steps.

`rank_suspicious_steps(step_results, detected_errors)`

Produces a ranked list of failing steps. Uses `_resolve_root()` to compute each failure's distance to its chain root by following `caused_by_map` links upward (with cycle guard). Sorted by: confidence descending → distance ascending → step index ascending. Root causes at distance 0 are most actionable; downstream symptoms have larger distances.

`build_diagnosis_report(step_results, detected_errors, active_failure_map, total_steps, root_cause_report=None)`

Assembles the full final report with four sections:

- **Section A — Failure chain analysis:** Mechanical chains from `build_failure_chains()`
- **Section B — Root cause chain analysis:** LLM-identified chains from `RootCauseReport`; renders `chain_links` with hop distances computed by `_resolve_root()` on the LLM-identified graph, plus `corrupted_state`, `chain_summary`, `downstream_impact`, and `recommended_investigation`
- **Section C — Suspicious step ranking:** From `rank_suspicious_steps()`
- **Section D — Summary statistics:** Total/analyzed/skipped/passed/failed, confidence distribution by tier, failure type breakdown, average confidence

load_project_data(folder_path)

Entry point for loading a test trace. Calls `find_fail_json()` to locate the input file, tries three encodings (utf-8, utf-8-sig, cp1254), then `detect_input_format()` to classify it. Returns a normalized flat list of step dicts.

6.2 agents.py — AI Agents

Role: Implements all four AI agents as Python functions, plus LangChain `RunnableLambda` wrappers for the three step-level agents and a direct function for the post-run Root Cause Analysis Agent.

API Client Management

```
def _get_client(force_new: bool = False) -> OpenAI:
```

Lazy-initializes the OpenAI client from the `OPENAI_API_KEY` environment variable. `force_new=True` re-creates the client, clearing stale TCP sockets — required for Google Colab's unstable network environment.

Retry Logic

```
def _api_call_with_retry(call_fn, max_retries: int = 3):
```

Wraps every API call with retry + exponential backoff (2s, 4s between attempts). Retries on:

- `OSError` errno 107 (`ENOTCONN`) or 111 (`ECONNREFUSED`) — Colab socket disconnection
- API transient errors: `rate limit`, `503`, `502`, `timeout`, `connection`

Non-retriable errors (authentication failures, malformed requests) propagate immediately.

Agent 1 — Vision Observer (`run_vision_observer`)

Sends PREV and POST screenshots to GPT-4o as inline base64 `image_url` blocks with `detail="high"` for accurate OCR. Handles both disk-based images (v33 format) and inline base64 images (G411 format):

```
if inputs.get("prev_b64"):
    prev_b64 = inputs["prev_b64"]
    prev_mime = detect_mime_from_b64(prev_b64)
else:
    prev_b64 = _encode_image_base64(inputs["prev"])
    prev_mime = _image_mime_type(inputs["prev"])
```

Model call: `gpt-4o, temperature=0, max_tokens=500, response_format={"type": "json_object"}`. Returns `VisionObservation` or a safe default on parse failure.

Agent 2 — Logic Decider (`run_logic_decider`)

Text-only GPT-4o call using `response_format={"type": "json_object"}`. The prompt implements a **two-phase structure**:

- **Phase 1 (Verdict):** Apply decision rules in strict priority order based on visual and log evidence. Commit to `is_failure` before reading prior history.
- **Phase 2 (Root Cause Enrichment):** Consult the last 3 active failure summaries to write a contextualized `root_cause`. These are placed *after* the decision rules to prevent anchoring bias.

Log evidence (`[LOG EVIDENCE]` block + `[LOG EVIDENCE RULES]`) is only injected when `log_report.has_logs=True`. When no logs are available, the prompt is byte-for-byte identical to a log-less invocation.

Model call: `gpt-4o, temperature=0, max_tokens=450, response_format={"type": "json_object"}`. Falls back to a system_error `FinalDecision` on parse failure.

Agent 3 — Log Observer (`run_log_observer`)

GPT-4o-mini call that parses sliced log text into `LogObservation`. Accepts `log_text` (pre-sliced) or `log_file_path` (legacy). Pre-processes log text to normalize uppercase level values (`"INFO" → "info"`, `"DEBUG" → "info"`) before sending, preventing Pydantic validation failures from GPT-4o-mini's tendency to echo raw casing.

Post-processing via `_normalize_log_obs_dict()` coerces output in-place:

- `level` and `source` normalized to valid `Literal` values
- `latency_ms` coerced to `float` (strips `"ms"` suffix, handles `null`)
- `status_code` coerced to `int`

Model call: `gpt-4o-mini, temperature=0, max_tokens=600, response_format={"type": "json_object"}`. GPT-4o-mini is sufficient for structured log extraction at a fraction of the cost of GPT-4o.

Agent 4 — Root Cause Analysis Agent (`run_root_cause_agent`)

A **post-run GPT-4o call** executed once after all steps complete. It receives:

- The full sequence of all step results (to understand execution context)
- Detailed failure entries (instruction, expected behavior, observed behavior, root cause)
- The mechanical suspicious step ranking as a preprocessing hint

Unlike the per-step agents, this agent reasons across the entire run simultaneously. It can identify semantic causal chains that span many steps and that `find_likely_cause()` misses due to temporal decay thresholds.

```
def run_root_cause_agent(step_results: list, detected_errors: list) ->
    RootCauseReport | None:
    if not detected_errors:
        return None
    # ... builds preprocessing hint from rank_suspicious_steps() ...
    # ... sends full context to GPT-4o ...
```

```
# ... validates root_step is an actual failure index ...
# ... returns RootCauseReport with chain_links, root_step, and narrative
fields
```

The function validates that all step indices in `chain_links` and `root_step` are actual failure indices before returning. Returns `None` on any error (the report renders gracefully without it).

Model call: `gpt-4o, temperature=0, max_tokens=700, response_format={"type": "json_object"}`.

RunnableLambda Wrappers

```
vision_chain = RunnableLambda(run_vision_observer)
logic_chain  = RunnableLambda(run_logic_decider)
log_chain    = RunnableLambda(run_log_observer)
```

These wrappers make the three per-step agents callable via `.invoke()`, consistent with the LangGraph node interface. `run_root_cause_agent` is called directly (not as a runnable) since it executes once outside the per-step loop.

6.3 `graph.py` — LangGraph Orchestration

Role: Defines and compiles the six-node LangGraph `StateGraph`, implements each node as a Python function, and manages the unified log store.

Graph Structure

```
def build_graph():
    workflow = StateGraph(GraphState)

    workflow.add_node("router",    router_node)
    workflow.add_node("skip",      skip_node)
    workflow.add_node("vision",    vision_node)
    workflow.add_node("log",       log_node)
    workflow.add_node("decider",   decider_node)
    workflow.add_node("accumulate", accumulate_node)

    workflow.set_entry_point("router")
    workflow.add_conditional_edges("router", route_step, {
        "skip": "skip", "analyze": "vision", "end": END,
    })
    workflow.add_edge("vision",    "log")
    workflow.add_edge("log",       "decider")
    workflow.add_edge("decider",   "accumulate")
    workflow.add_edge("skip",      "router")
    workflow.add_edge("accumulate", "router")
    return workflow.compile()
```

Node Descriptions

Node 1 — `router_node`

Reads the current step from `steps_data`, calls `extract_step_context()` to normalize it, and determines the route:

- "end" if all steps have been processed → calls `run_root_cause_agent()` then `build_diagnosis_report()`
- "skip" if the step has `action="wait"`, `requiresVLM=False`, `element="null"`, or missing screenshots
- "analyze" otherwise

The "end" path is the only place `run_root_cause_agent()` is called, ensuring it receives the complete picture of all failures.

Node 2 — `skip_node`

Increments `current_step_index` without calling any AI agent. Logs a skip reason (`images_missing` or `vlm_skip`) to the console.

Node 3 — `vision_node`

Resolves image inputs via `_get_image_inputs()`, calls `vision_chain.invoke()`, and stores the `VisionObservation` dict in `_current_vision_obs`. On API error, stores a safe fallback.

Node 4 — `log_node`

Implements the log source priority chain:

1. G411 inline logs (`_inline_console_logs`, `_inline_network_logs`)
2. Pre-assigned entries from `_parsed_logs.step_map` (mode "preassigned")
3. Dynamic keyword/positional slicing via `extract_step_logs()` (mode "dynamic")
4. Legacy per-step file (`step_N_logs.json`)

Calls `log_chain.invoke()` with the relevant log text. Prints diagnostic flags (`NET_FAIL`, `SYS_ERR`, `LATENCY`) when found.

Node 5 — `decider_node`

Reconstructs `VisionObservation` and `LogObservation` from the state dicts, then calls `logic_chain.invoke()` with the full evidence package. Stores the `FinalDecision` dict in `_current_decision`.

Node 6 — `accumulate_node`

The most complex node. Responsibilities:

1. Post-hoc confidence adjustment:

- Log parsing failed → `confidence -= 0.10`
- No logs at all → `confidence -= 0.05`
- Minimum floor: `0.10`

2. **Causal chain gating:** Calls `find_likely_cause()` only when `confidence >= 0.75` OR `has_network_failure` or `has_system_error`. Low-confidence failures are recorded but cannot propagate spurious mechanical chains.
3. **State accumulation:** Appends step result to `step_results`, updates `detected_errors`, `active_failures`, `active_failure_map`, and increments `current_step_index`.

Unified Log Store

```
_parsed_logs: ParsedLogs | None = None

def set_log_source(parsed: ParsedLogs) -> None:
    global _parsed_logs
    _parsed_logs = parsed

def set_unified_logs(logs: list) -> None: # legacy shim
    global _parsed_logs
    _parsed_logs = ParsedLogs(mode="dynamic", format_name="legacy_raw",
raw_entries=logs or [])
```

The module-level `_parsed_logs` is intentionally not serialized into `GraphState`. LangGraph JSON-serializes state on every step; a large log file would make this prohibitively expensive. `set_log_source()` is called once before `app.invoke()`.

Image Utilities

`find_image(folder, step_num, suffix)` tries `.png`, `.jpg`, and `.jpeg` extensions in order. `_get_image_inputs()` transparently selects between inline base64 (G411) and disk-based (v33) image delivery.

6.4 `input_adapter.py` — Format Detection and Normalization

Role: Makes the pipeline format-agnostic by detecting the input format and converting all inputs into the canonical v33 flat step format before they reach the graph.

`detect_input_format(data)`

Returns `"g411"` or `"v33"`. The G411 fingerprint requires all three conditions:

1. Top-level dict with a `stepDetails` list
2. Each item's inner `step` object has only `action`, `stepInstruction`, and optionally `condition` keys — no `step_idx`
3. Each item carries `prevScreenshot` and `currScreenshot` base64 fields

Any deviation returns `"v33"`.

`normalize_g411_steps(data)`

Converts G411 `stepDetails` into canonical flat dicts. Key transformations:

- **Action mapping:** `"verify"` → `"abstractVerification"`, `"type"` → `"click"`
- **Element fallback:** G411 steps have no `element` field; `stepInstruction` is used as proxy
- **Inline delivery:** Base64 images stored under `_inline_prev_b64/_inline_curr_b64`; log text under `_inline_console_logs/_inline_network_logs`
- **Step index:** Array index `idx` is assigned as `step_idx`

`find_fail_json(folder_path)`

Searches for the trace file in this order:

1. `steps.json` (v33 convention)
2. `fail.json` (v33 alternative)
3. Glob `*fail*.json` (G411 naming: `passo_fail.json`, `bank-app-fail.json`)

Returns the absolute path of the first match, or `None`.

`detect_mime_from_b64(b64)`

Infers image MIME type from the first bytes of the base64 string:

- `/9j...` → `image/jpeg` (JPEG magic bytes `FFD8`)
- Anything else → `image/png`

6.5 `parser/log_parser.py` — Unified Log Ingestion

Role: Parses a `log.json` file — regardless of its internal format — into a single `ParsedLogs` dataclass that the graph consumes uniformly. Supports four distinct log shapes encountered across the 12 evaluated applications.

`ParsedLogs` Dataclass

```
@dataclass
class ParsedLogs:
    mode: Literal["preassigned", "dynamic", "empty"]
    format_name: str
    step_map: Dict[int, List[dict]] # used when mode == "preassigned"
    raw_entries: List[dict]        # used when mode == "dynamic"
    warnings: List[str]
```

- **preassigned:** Step ownership determined at parse time; `log_node` does a direct `step_map.get(step_idx, [])` lookup.
- **dynamic:** Step ownership inferred at query time by `extract_step_logs()`.
- **empty:** No log file found or file unreadable.

The `warnings` list is surfaced by `runner.py` so index mismatches are visible at startup rather than silently corrupting step assignment.

Format Detection — `_detect_format(raw)`

Inspects the first entry to classify:

- `step` field is a dict → `nested`
- `step` field is an int → `flat_indexed`
- Contains `StepRunner` entries with "Step N passed/failed" messages (first 20 entries scanned) → `flat_assertions`
- Otherwise → `raw`

Format A — `flat_indexed`

Flat array of entries each carrying `step: <int>`. **1-indexed shift detection:** if the minimum observed step is 1 and no step 0 exists, all keys are shifted down by 1. This correctly handles logs where step numbering starts at 1 rather than 0 (e.g., QNB app). The shift does not require `max_step == total_steps` — trailing skipped/unlogged steps make `max_step < total_steps` even in genuinely 1-indexed logs. Out-of-range indices after shift are reported as warnings.

Format B — `flat_assertions`

Flat array with no `step` fields; boundaries defined by `source: "StepRunner"` entries containing "Step N passed/failed" messages. All entries accumulated before each marker are assigned to that step. Trailing entries (after the final marker) are orphaned with a warning. A mismatch between marker count and `total_steps` triggers a data-quality warning about possible log-test misalignment.

Format C — `nested`

Array where each item is `{step: {action, stepInstruction}, status, logs?: str}`. Array index is the step index. The `logs` string is pipe-delimited (`key: value | key: value | ...`) and parsed by `_parse_pipe_delimited()` into a list of entry dicts. Missing `logs` fields produce empty buckets with a warning.

Format D — `raw`

Flat, unstepped, heterogeneous entries. Returns `mode="dynamic"` so that `log_node` delegates to `extract_step_logs()` for keyword+positional assignment. `vlm_agent` source entries are excluded during keyword scoring to prevent contamination.

6.6 `runner.py` — Colab Entry Point

Role: The single script that runs in Google Colab to execute the full analysis pipeline. Handles dependency installation, authentication, path configuration, log ingestion, and graph invocation.

Startup Sequence

```
!pip install -q openai langchain langchain-core "langgraph>=0.2,<0.3" pydantic
nest_asyncio
```

```

import nest_asyncio
nest_asyncio.apply() # Required: allows asyncio in Colab's existing event loop

PROJECT_PATH = "/content/drive/MyDrive/Grad Project"
STEPS_PATH = f"{PROJECT_PATH}/steps"

# Add both project root and parser/ sub-package to sys.path
for _p in (PROJECT_PATH, os.path.join(PROJECT_PATH, "parser")):
    if _p not in sys.path:
        sys.path.insert(0, _p)

if "OPENAI_API_KEY" not in os.environ:
    os.environ["OPENAI_API_KEY"] = input("OpenAI API Key: ")

```

Graph Invocation

```

steps_data = load_project_data(STEPS_PATH)
_parsed_logs = parse_log_file(os.path.join(STEPS_PATH, "log.json"),
total_steps=len(steps_data))
set_log_source(_parsed_logs)

# Prints log parser diagnostics before starting
print(f"Log parser: format={_parsed_logs.format_name}, mode={_parsed_logs.mode}")
for _w in _parsed_logs.warnings:
    print(f" [WARN] {_w}")

res = app.invoke({
    "current_step_index": 0,
    "steps_data": steps_data,
    "trace_files_path": STEPS_PATH,
    "final_report": "", "stop_execution": False,
    "detected_errors": [], "active_failures": [],
    "active_failure_map": {}, "step_results": [],
    "_current_ctx": None, "_current_vision_obs": None,
    "_current_log_obs": None, "_current_decision": None,
    "_route": None, "_skip_reason": None,
}, config={"recursion_limit": 150})

```

The `recursion_limit: 150` allows analysis of datasets up to approximately 70 VLM-processed steps (each step visits the router + analysis nodes + accumulate = ~2 graph hops per step).

`download_results(results_dir=None)`

A Colab-specific utility that triggers browser download of all `.txt` and `.json` files in the `results/` directory. Gracefully exits if not running in Colab.

7. Agent Prompting Methodology

7.1 Vision Observer Prompt Design

The Vision Observer receives a multi-modal prompt: two images as base64 `image_url` blocks with `detail="high"`, followed by a structured text prompt with three tasks.

Key design principles:

Separation of observation and judgment. The observer is explicitly prohibited from making pass/fail decisions. This prevents anchoring bias in the Logic Decider.

Priority-ordered visual checks. The observer checks in this order:

- 1. **Target state changes** — any visual or content change to the target element (color, size, enabled/disabled, text/value updated). Includes a new **BUTTON DISMISSED** clause: if the target was present in PREV and is completely absent in POST while the screen shows content/results, the button completed its function and dismissed itself. Reports as `"Target state changed: [button name] dismissed from POST after activating its function."`
- 2. **Screen context changes** — different screen title, navigation bar label, app header, address bar content. Extended in the latest version to cover address bar and status bar changes explicitly.
- 3. **New content sections** — lists, cards, form fields, price panels appearing in POST.
- 4. **General element presence** — target in PREV only / POST only / Both / Neither.

Semantic equivalence. The observer matches targets by meaning, not exact text — a "close" button, an X icon, and "kapat" in Turkish are the same dismiss control. However, specific data values (times, prices, dates) must match character-by-character.

Strict value OCR gating. The `VALUE NOT CONFIRMED IN POST` phrase is restricted to `abstractVerification/verify` steps that explicitly ask to confirm a specific data value. It never appears for `click`, `waitUntil`, `wait`, or `type` steps.

The output is constrained to exactly three JSON keys: `prev_screen_elements`, `post_screen_changes`, `semantic_screen_summary`. `max_tokens=500`.

7.2 Logic Decider Prompt Design

The Logic Decider receives a text-only prompt containing the full evidence package and applies a priority-ordered decision rule set.

Two-phase structure. Phase 1 (Verdict): read evidence, apply rules, commit to `is_failure`. Phase 2 (Root Cause): consult prior failure history to enrich `root_cause`. This sequencing prevents prior failures from anchoring the verdict.

Priority-ordered decision rules (in order):

Priority	Rule	Condition	Verdict
0	System Overlay Exception	System-level dialog over main screen	SUCCESS
0B	App Loading Exception	Click → loading spinner overlay (no interactive elements) in POST	SUCCESS

Priority	Rule	Condition	Verdict
0C	Target State Change Success	Observer reports "Target state changed" (any visual/content change) OR target selected/active in POST but not PREV	SUCCESS
1	Target Absent	Element missing from both PREV and POST	<code>element_missing</code>
1 (Close/Dismiss)	Close/Dismiss Exception	Dismiss control in PREV, gone in POST	SUCCESS
1 (Navigation)	Navigation Exception	Target in PREV, absent in POST, screen confirms forward navigation	SUCCESS
1 (Wrong-Item)	Wrong-Item Exception	Click step, target absent from PREV, but POST shows navigation/overlay	<code>content_mismatch</code>
1 (Wrong-Value)	Wrong-Value Navigation	Click on data-value target, POST shows different value in detail view	<code>content_mismatch</code>
2	Strict Value Verification	<code>abstractVerification</code> step with explicit data value in instruction	<code>content_mismatch</code> if value not confirmed
3	Success	No failure condition matched	SUCCESS
3 (Content Expansion)	Content Expansion Success	Observer reports "New content appeared"	SUCCESS
3 (Trigger-Dismiss)	Trigger-Dismiss Success	Observer reports "Target state changed: [button] dismissed from POST"	SUCCESS

New in latest version:

- **Decimal separator insensitivity** added to Rule 2: `5250,00 == 5250.00` (Turkish locale uses comma as decimal separator). If the Observer describes the value without exact punctuation but the amount matches, it passes.
- **TRIGGER-DISMISS SUCCESS** (Rule 3) added as a secondary safety net for the `BUTTON DISMISSED` clause introduced in the Vision Observer.

Log evidence fusion. Injected only when `has_logs=True`. Log evidence can only override PASS→FAIL (for server-side failures invisible to the UI); it can never override FAIL→PASS. The **Gesture-Confirmation Rule** prevents gesture-only log entries (click registered) from being treated as evidence that the app responded. The **Wait-Until Log Exception** allows log evidence to override visual absence for polling-based wait conditions when the log explicitly confirms the condition was met.

Confidence calibration. Model uses the full 0.0–1.0 range with explicit tier definitions and is warned against anchoring to 0.9.

7.3 Log Observer Prompt Design

The Log Observer uses GPT-4o-mini with `max_tokens=600`. The prompt includes step context for relevance filtering, up to 3000 characters of sliced log text, explicit field mapping instructions for nested log objects, source normalization rules, and precise boolean flag definitions.

The model is cheaper than GPT-4o and sufficient for structured information extraction from semi-structured log text. It is not asked to make judgments — only to extract and classify.

7.4 Root Cause Analysis Agent Prompt Design

The Root Cause Analysis Agent receives a comprehensive prompt with three data sections:

[ALL STEPS – EXECUTION SEQUENCE]: Every step with its verdict (FAIL or pass), action type, and instruction. This gives the LLM the full execution context — it can understand what came before and after each failure.

[FAILURES – DETAILED]: For each failing step: instruction, expected behavior, observed behavior, root cause (from Logic Decider), failure type, and confidence. This is the primary input for reasoning.

[PREPROCESSED SUSPICIOUS STEP RANKING – mechanical hint only]: The output of `rank_suspicious_steps()` provided as a starting point. The prompt explicitly notes that this ranking "only captures chains where the scoring threshold was met" and that the LLM "is NOT limited by it." This is critical: the LLM can identify long-gap semantic chains (e.g., Step 7 → Step 14) that the mechanical `find_likely_cause()` misses due to temporal decay.

The LLM is asked to:

1. Identify the true root failure step
2. Build `chain_links` for all causal connections, including those missed by the mechanical ranking
3. Describe the corrupted application state
4. Explain how the corruption propagated
5. Recommend what to investigate first

After receiving the response, `_resolve_root()` computes exact hop distances on the LLM-identified graph, providing consistent distance metrics in the report regardless of the chain structure.

8. Cascading Failure Analysis

A key innovation is linking related failures into causal chains rather than reporting them as independent events.

Mechanical Chain Detection (Per-Step)

`find_likely_cause()` runs in `accumulate_node` after each failure, subject to the confidence gate (≥ 0.75 or hard log evidence). It uses the `_TYPE_CORRELATION` matrix and element overlap to identify the most likely upstream cause. The result is stored as `caused_by_step` in the failure record.

`build_failure_chains()` groups failures by traversing these `caused_by_step` links. Isolated failures (no upstream cause) are reported separately from multi-step chains.

LLM-Based Chain Detection (Post-Run)

`run_root_cause_agent()` complements the mechanical approach by reasoning across all failures simultaneously. It can identify:

- **Long-gap chains** (e.g., Step 7 → Step 14) where temporal decay reduces the mechanical score below threshold
- **Semantic chains** where the connection is in the application flow logic rather than shared element text
- **Cross-type chains** that the `_TYPE_CORRELATION` matrix underweights

The LLM returns `chain_links` as a directed graph. `build_diagnosis_report()` renders this graph using `_resolve_root()` to compute exact hop distances, then displays the root, downstream steps with their distances, and narrative fields (`corrupted_state`, `chain_summary`, `downstream_impact`, `recommended_investigation`).

Causal Chain Gating

`find_likely_cause()` is only called from `accumulate_node` when:

- `confidence >= 0.75` (high-confidence failure), **OR**
- `has_network_failure` or `has_system_error` is true (hard log evidence)

This gate prevents low-confidence false positives — often caused by mid-animation screenshots or ambiguous presence checks — from propagating spurious mechanical causal chains downstream. Note: `run_root_cause_agent()` does not have this gate — it reasons across all failures regardless of their mechanical chain status.

9. LLM-Powered Root Cause Chain Analysis

This is the fourth agent and a key differentiator from the mechanical `find_likely_cause()` approach.

Why it is needed: `find_likely_cause()` scores below 0.3 for failure pairs that are separated by many steps or whose element text differs. In the Yemeksepeti case, Step 7 (wrong pizza selected) and Step 14 (checkout confirms wrong pizza) are 7 steps apart and differ in action type (`click` vs. `abstractVerification`), causing the mechanical scorer to miss the link. The LLM agent can reason about this at the application-flow level.

How it works:

1. The complete execution history is presented alongside all failures with full diagnostic context.
2. The mechanical ranking is provided as a non-binding hint ("here is what we found mechanically, but you are not limited by it").
3. The LLM produces `chain_links` (a directed graph), `root_step`, and narrative fields.
4. The code validates `chain_links` and `root_step` against the actual set of failure step indices.
5. `_resolve_root()` computes hop distances on the LLM-identified graph for consistent reporting.

Report output example:

```
ROOT CAUSE CHAIN ANALYSIS
```

```
=====
```

```
Root: Step 7 [content_mismatch | conf: 0.95]
```

```
└─ Step 8 [content_mismatch | dist: 1 hop ] Verification confirms 21:15 not
```

```

12:15...
└─ Step 14 [content_mismatch | dist: 2 hops ] Checkout order shows wrong
pizza...

Corrupted state:   Selected flight item in session (wrong flight stored after Step
7)
Chain summary:     Step 7 clicked the wrong flight row; the session stored 21:15
instead of
                   12:15, and all downstream verification steps confirmed the
wrong data.
Downstream impact: All abstractVerification steps that check departure time fail
because
                   the stored value is incorrect.
Investigate first: The flight selection click handler – verify scroll position and
row
                   tap target boundaries.
Confidence:        0.92

```

10. Suspicious Step Ranking

The `rank_suspicious_steps()` function provides an **actionable prioritization** for engineers. Each failing step is annotated with its distance to its chain root — root causes (distance 0) are the most actionable; downstream symptoms have larger distances.

Sorting: Confidence descending → distance ascending → step index ascending.

Report output example:

```

SUSPICIOUS STEP RANKING
=====
Steps ranked by confidence (descending). Distance = hops to chain root.

#1 Step 7 | conf: 0.95 | type: content_mismatch | dist: 0 (root)
| Click on 12.15→13.35 selected wrong fli...
#2 Step 4 | conf: 0.90 | type: element_missing | dist: 0 (isolated)
| Günlük button no state change
#3 Step 8 | conf: 0.88 | type: content_mismatch | dist: 1 -> root:
Step 7 | Verification confirms 21:15 not 12:15

```

This ranking helps engineers immediately identify where to start debugging: the highest-confidence root cause at the top, downstream symptoms below.

11. Confidence Scoring and Calibration

Confidence is tracked at multiple levels and used for both causal chain gating and report statistics.

Self-Assessed by Logic Decider

The model outputs a **confidence** float (0.0–1.0) based on evidence quality. Tier definitions:

- **0.90–1.00:** All evidence strongly agrees (clear visual signal, logs confirm, no ambiguity)
- **0.70–0.89:** Mostly agrees, minor ambiguity (partial text match, log absent but visual clear)
- **0.50–0.69:** Notable ambiguity (conflicting signals, semantic equivalence applied)
- **0.30–0.49:** High uncertainty (missing images, system overlay, ambiguous UI state)

Post-Hoc Adjustment in **accumulate_node**

Condition	Adjustment
Log parsing failed (log_summary starts with "Log parsing")	−0.10
No log data at all (has_logs=False)	−0.05
Minimum floor	0.10

Used For

1. **Causal chain gating:** **find_likely_cause()** only called when **confidence** ≥ 0.75
2. **Suspicious step ranking:** Primary sort key
3. **Report statistics:** High/medium/low tier counts, average confidence across all analyzed steps

12. Technology Stack and Design Choices

Locked Constraints

Layer	Choice	Rationale
Language	Python 3.10+	Colab compatibility, union type hints (X Y)
Data validation	Pydantic v2	All agent I/O strictly typed; no raw dicts
LLM framework	LangChain	Ecosystem integration; RunnableLambda interface
Agent orchestration	LangGraph $\geq 0.2, < 0.3$	Stateful sequential graph; auditable execution paths
Vision model	GPT-4o	State-of-the-art multimodal OCR; no GPU required
Logic model	GPT-4o	Best reasoning quality; text-only
Root cause model	GPT-4o	Complex cross-step reasoning
Log observer	GPT-4o-mini	Sufficient for structured extraction; $\sim 10\times$ cheaper
API client	OpenAI Python SDK	Lazy-initialized; retry with exponential backoff
Image encoding	base64 stdlib	Inline delivery; no external hosting needed
Runtime	Google Colab (any)	CPU runtime sufficient; no GPU packages

Dependencies

```
pip install openai langchain langchain-core "langgraph>=0.2,<0.3" pydantic nest_asyncio
```

No torch, transformers, bitsandbytes, qwen-vl-utils, or HuggingFace packages required.

Why LangGraph Over AutoGen

- 1. **Determinism:** Academic fault localization requires reproducible, traceable execution paths. LangGraph's explicit typed graph is auditable. AutoGen's conversational loops are non-deterministic.
- 2. **Sequential stateful processing:** Each step's verdict and failure context must feed downstream steps. LangGraph's `GraphState` accumulates this naturally.
- 3. **Conditional routing:** `add_conditional_edges` handles skip/analyze/end routing cleanly.
- 4. **LangChain ecosystem:** Already a hard constraint; LangGraph is LangChain's first-party orchestration layer.

13. Evaluation Results

Dataset

Dataset	Apps	Total Steps	Skipped	Analyzed	Ground Truth Failures
Our Traces	7	90	5	85	12
Group 411	5	46	0	46	13
Combined	12	136	5	131	25

Performance Metrics

Dataset	TP	FP	FN	Precision	Recall	F1
Our Traces (7 apps)	12	0	0	100.0%	100.0%	100.0%
Group 411 (5 cases)	11	1	2	91.7%	84.6%	88.0%
Combined	23	1	2	95.8%	92.0%	93.9%

Applications Tested

Our Traces (7 apps):

App	Steps	Failures	Log Format	Notes
Migros	8	1	nested (preassigned)	POST /api/v1/cart/confirm → 400 Bad Request
Bitaksi	11	0	flat_assertions (preassigned)	All steps passed

App	Steps	Failures	Log Format	Notes
Clock	7	0	flat_indexed (preassigned)	1-indexed log; auto-shifted
Flight App	26	6	raw (dynamic)	Wrong flight selected → cascade
QNB	8	1	flat_indexed (preassigned)	Günlük button no UI response
Yemeksepeti	17	2	raw (dynamic)	Wrong item selected → checkout confirms wrong item
O Bilet	13	2	raw (dynamic)	POST /api/v3/bus/reserve-seat → 504 Gateway Timeout

Group 411 (5 cases):

App	Steps	GT Failures	TP	FP	FN
Passo App	~10	1	1	1	0
Bank App	~10	3	2	0	1
Ticket App	~12	4	4	0	0
Rent-a-Car	~10	4	3	0	1
Wikipedia	~4	1	1	0	0

Ground Truth Failures

Our Traces (12 failures):

App	Step	Failure	Evidence
Migros	7	POST /api/v1/cart/confirm → 400 Bad Request; cart state inconsistency	Log: status: FAILED + network ERROR
Flight App	18	Click on 12.15→13.35 selected wrong flight (21:15)	Visual: screen shows 21:15 departure in POST
Flight App	20–24	Cascading content_mismatch — all verification steps confirm 21:15 instead of 12:15	Visual: expected flight time never appears
QNB	4	Click on Günlük option produced no UI state change	Visual: no change between PREV/POST
Yemeksepeti	7	Click on Mixed (Medium) Pizza opened Mediterranean (Central) detail page	Visual + Log: wrong item in POST
Yemeksepeti	14	AbstractVerification confirmed wrong pizza persists to checkout	Visual: Mediterranean (Central) in order summary

App	Step	Failure	Evidence
O Bilet	7	<code>POST /api/v3/bus/reserve-seat</code> → 504 Gateway Timeout	Log: NET_FAIL + SYS_ERR
O Bilet	8	Navigation to PassengerDetailsActivity blocked by failed reservation	Log: SYS_ERR + UI_STALL

Group 411 (13 failures):

App	Step	Failure	Evidence
Passo App	10	Click on occupied (black) seat — no state change	Visual: no change between PREV/POST
Bank App	4	<code>POST /transfer</code> → 500 Internal Server Error; transfer silently failed	Log: NET_FAIL + SYS_ERR
Bank App	6	Balance 5250.00 TL unchanged after "successful" transfer	Cross-step state: balance should update
Bank App	7	<code>Bakiyeyi doğrula</code> element not visible in PREV or POST	Visual: element absent
Ticket App	2	Click on <code>Theater</code> category — no UI response	Visual: no change
Ticket App	3	<code>RenderFlex overflow by 869 pixels</code> on Sports Events screen	Log: rendering exception in console
Ticket App	6	Click on occupied seat A3 — <code>Seat unavailable</code>	Log: SYS_ERR; Visual: no change
Ticket App	10	<code>POST /api/v1/payment</code> → 500 Internal Server Error	Log: NET_FAIL + SYS_ERR
Rent-a-Car	5	<code>Kirala</code> for Mercedes-Benz G-Class blocked — corporate account required	Log: SYS_ERR
Rent-a-Car	8	Promo code UI showed discount applied but total remained ₺17,000	Visual: amount unchanged
Rent-a-Car	9	Discount message shown but total ₺17,000 unchanged	Visual: content mismatch on amount
Rent-a-Car	10	<code>POST /payment</code> → 504 Gateway Timeout on Izmir regional gateway	Log: NET_FAIL + SYS_ERR
Wikipedia	4	Target instruction not present in any visible UI element	Visual: element absent

Error Analysis

False Negative 1 — Bank App Step 6 (cross-step state tracking): The balance after a "successful" transfer remained unchanged at 5250.00 TL. The system correctly detected the underlying transfer failure (Step 4: 500 Internal Server Error) but did not flag the static balance on the verification screen. Requires a `state_scratchpad` in `GraphState` tracking numerical values from prior steps.

False Negative 2 — Rent-a-Car Step 8 (price-discount inconsistency): After a promo code was applied and the UI showed a success message, the total remained ₺17,000. The Observer failed to cross-check the total against the claimed discount. Requires a specific Observer prompt clause for the "trigger-reveal" pattern.

False Positive 1 — Passo App Step 5 (content expansion misread): The Observer misread a ticket category list appearing after a button click as the button becoming grayed-out/deactivated. Addressed in the latest version with the explicit **CONTENT EXPANSION SUCCESS** clause in Rule 3 and **BUTTON DISMISSED** handling in the Observer.

14. Known Limitations and Future Work

Open Issues

Cross-step state propagation. No mechanism exists to compare a value observed in one step against a value derived from a prior step. Fix: a `state_scratchpad` dict in `GraphState` storing key numerical values from `abstractVerification` passes, injected as `[STATE CONTEXT]` for downstream verifications.

Price-discount consistency check. When a promo code step shows a discount success message alongside an unchanged total price, the Observer should detect the inconsistency. Fix: a specific Observer prompt clause for trigger-reveal patterns where a success message appears alongside an unchanged numerical value.

Confidence miscalibration at high confidence. Three incorrectly evaluated steps had high confidence (≥ 0.85). Potential mitigations: penalize when `has_logs=False` for action types that typically generate backend events; penalize when `semantic_screen_summary` contradicts `post_screen_changes` at the element level.

Long-gap semantic chains in mechanical scorer. `find_likely_cause()` scores below 0.3 for failures separated by more than ~4 steps when failure types differ. The LLM Root Cause Analysis Agent addresses this post-hoc, but real-time linking (in `accumulate_node`) would require keyword similarity between the current instruction and prior `root_cause` strings to override the distance-based threshold.

Architectural Improvements

Parallel step processing. Independent steps that do not share UI elements with prior failures could be analyzed in parallel, reducing total runtime.

Adaptive log slicing. The keyword scoring in `extract_step_logs()` uses equal weights. A learned weighting based on action type and historical hit rates could improve assignment accuracy.

Multi-run comparison. Running the analysis over multiple test runs of the same scenario and comparing results would reduce false positives from transient UI states (loading screens, animations) captured mid-frame.

File Structure Summary

```

Grad Project/
├── models.py      # Pydantic models, GraphState, all helper functions
                   # VisionObservation, FinalDecision, LogObservation
                   # ChainLink, RootCauseReport (new)
                   # extract_step_context, robust_json_extract
                   # _normalize_keywords, extract_step_logs
                   # find_likely_cause, _TYPE_CORRELATION matrix
                   # build_failure_chains, rank_suspicious_steps
                   # _resolve_root (cycle-safe hop distance)
                   # build_diagnosis_report (4 sections, RCA-aware)
                   # load_project_data, format_active_failures

├── agents.py      # OpenAI API agents + RunnableLambda wrappers
                   # _get_client (lazy-init, force_new for retry)
                   # _api_call_with_retry (3 attempts, exponential backoff)
                   # _encode_image_base64, _image_mime_type
                   # run_vision_observer → GPT-4o, vision, max 500 tokens
                   # run_logic_decider  → GPT-4o, text,   max 450 tokens
                   # run_log_observer   → GPT-4o-mini,   max 600 tokens
                   # run_root_cause_agent → GPT-4o, post-run, max 700 tokens
                   # _normalize_log_obs_dict (coercion for GPT-4o-mini

output)

├── graph.py       # LangGraph 6-node StateGraph
                   # router_node (context extraction, routing, report

generation)

                   # skip_node (non-VLM steps, images missing)
                   # vision_node (GPT-4o vision agent)
                   # log_node (log source priority chain)
                   # decider_node (GPT-4o logic agent)
                   # accumulate_node (confidence adjustment, causal gating)
                   # set_log_source, set_unified_logs (module-level log

store)

                   # find_image, _get_image_inputs (PNG/JPG/JPEG + inline
b64)

                   # build_graph() → compiled LangGraph app

├── input_adapter.py # Format detection and normalization (G411 ↔ v33)
                   # detect_input_format (fingerprinting logic)
                   # normalize_g411_steps (action mapping, inline delivery)
                   # find_fail_json (3-tier search: steps.json, fail.json,

glob)

                   # detect_mime_from_b64 (PNG vs JPEG from base64 header)

├── runner.py      # Colab entry point
                   # pip install, nest_asyncio, Drive mount, API key
                   # sys.path setup for PROJECT_PATH and parser/
                   # load_project_data → parse_log_file → set_log_source
                   # app.invoke with recursion_limit=150
                   # download_results() utility for Colab file download

```

```
└─ parser/
  └─ log_parser.py  # Unified log.json ingestion
                    # ParsedLogs dataclass (mode, format_name, step_map,
warnings)          # parse_log_file (entry point, auto-detects format)
                    # _detect_format (nested, flat_indexed, flat_assertions,
raw)               # _parse_flat_indexed (1-indexed shift detection)
                    # _parse_flat_assertions (StepRunner marker partitioning)
                    # _parse_step_nested (array-index step ownership)
                    # _parse_pipe_delimited (k: v | k: v row parser)
                    # raw format → dynamic mode (no parse-time assignment)
```

This report documents the Describer-Decider Engine as evaluated on 12 applications and 136 execution steps. Combined F1 score: 93.9%. Perfect precision and recall (100% F1) on all 7 proprietary test cases. The latest version adds an LLM-powered Root Cause Analysis Agent that reasons across all failures simultaneously, identifying long-gap semantic causal chains that the mechanical per-step scorer cannot detect.